

Homework 3: Threading and Load Tests!

New Attempt

- Due 2 Feb by 12:00
- Points 10
- Submitting a text entry box, a website url, a media recording, or a file upload

Part I: Time and Ordering!

This is a classic [paper](https://dl.acm.org/doi/10.1145/359545.359563)  <https://dl.acm.org/doi/10.1145/359545.359563>! Please post what you liked and didn't like about the paper on Piazza by Wednesday!

Part II: Thread Experiments Overview

A lot of the work in this course involves collecting data from systems to be able to assess tradeoffs and predict behaviour when a "party of nodes" is running. This week we are looking at the conceptual relationship between evidence you collect doing a few experiments with "a party of threads", and basic foundations of distributed systems. In this first part of the assignment, you will get some hands on experience with the basics of multithreading (*goroutines*) with Go. A nice resource for playing with Go is [GoByExample](https://gobyexample.com/)  <https://gobyexample.com/>. For each section below, bring some fun experimental results to your mock interview. There are questions here to guide you, but feel free to run other experiments and be creative! Super fun charts and graphs are always welcomed! Be ready to run your code, and show your group what you discovered!

For the handing portion of the assignment, please create a brief report of your experiments that

- includes screenshots of your results
- briefly describes the concepts you have learned, and the evidence you have uncovered that support those concepts!

Atomicity

Your first experiment focuses on the concept of atomic operations.

1. Follow the steps in [atomic counters in Go](https://gobyexample.com/atomic-counters)  <https://gobyexample.com/atomic-counters>
2. Run the exercise on your system (the GoPlayground is cool too, but it will be easier to rerun for your interview from your own system) and compare an *atomic* integer vs regular integer. *Note*, you may have to run it several times to see the race condition happen!

What values do you see? What is happening? Try running with the `-race` flag (described [here](https://go.dev/doc/articles/race_detector)  https://go.dev/doc/articles/race_detector). What does that do?

Collections

In this experiment, you will be writing to a *map* in Go. For every exercise, run tests at least 3 times and record the mean of the results.

Set up a plain `map[int]int`.

Spawn 50 goroutines (let's call each one "goroutine g ", where g ranges from 0 to 49).

In each goroutine, run a loop of 1,000 iterations (index it by i , from 0 to 999) and do:

```
m[g*1000 + i] = i
```

This writes 1,000 distinct key/value pairs into the *shared* map.

Wait for all 50 goroutines to finish, then print out `len(m)`.

You might see that your program crashed. Why? What's going on? (Remember that Claude Opus 4.1 in learning mode is encouraged here if this is all new to you!)

Mutex

Read a little about what a [mutex](https://pkg.go.dev/sync#Mutex) is and [this example](https://gobyexample.com/mutexes). Now try to "wrap" your map in a struct that contains a mutex. For every read and write to your hashmap, lock before that operation and unlock after. Print out the length of hashmap as well as total time taken for all your goroutines to finish. What's the lesson learned here?

RWMutex

In this experiment, replace mutex with [RWMutex](https://pkg.go.dev/sync#RWMutex). Print out the length and time taken. Did this change anything? Why or why not? What is the lesson learned here?

Sync.Map

For this experiment, you might want to read a little about [Sync.Map](https://pkg.go.dev/sync#Map). Now try to swap in `sync.Map` instead of your plain map:

```
var m sync.Map
// inside each goroutine:
m.Store(g*1000 + i, i)
// after all done, count entries with m.Range(...)
```

Run your experiment again, and document your results. What is the lesson learned now?

Make sure that you can summarize the tradeoffs in the different approaches above! More specifically, directly compare 3 sets of results you obtained for each tests above in some quantitative representation (this can be creative!). Explain the reasons behind these results. In this case, we are mostly writing to the map, but what will happen if read operations dominate? You might have observed that one of these is definitely the fastest, but what are the **tradeoffs between these approaches**?

File Access

In this experiment, you are finally moving into persistence. Goodness, is this also an issue?! Let's find out!

- **Create two write functions:** one that writes directly to the file on each loop iteration (unbuffered), and one that wraps the file in a `bufio.Writer` (buffered).

- **Unbuffered mode:** open (or create) your output file, record the start time, then loop—for example, 100,000 times—calling `f.Write([]byte(...))` once per line. After the loop, close the file and compute the elapsed time.
- **Buffered mode:** open the file again, wrap it in `bufio.NewWriter(f)`, record the start time, loop the same number of iterations using `w.WriteString(...)`, then call `w.Flush()`, close the file, and compute elapsed time.
- Print both durations back-to-back.

What? Why? What is going on and what are the lessons learned about tradeoffs from these results?

Context Switching

In this experiment you'll explore how Go switches between lightweight threads (goroutines) and we want to think about this cost a little.

Start by configuring the Go runtime to use just one operating-system thread:

```
runtime.GOMAXPROCS(1)
```

Then, launch two goroutines that simply pass a small, empty signal back and forth over an unbuffered channel 1 million times (again, feel free to get Claude to help if this is all new to you, and learning mode is helpful here!). Measure the total duration of all those hand-offs, then divide by twice the number of round-trips to find an average "switch time."

Next, remove the one-thread restriction so Go can schedule goroutines on multiple threads and repeat the experiment—again timing the same ping-pong loop.

Record and compare the two averages. Which one is faster? Why? How do you think this relates to the cost of context switching between processes, containers, and virtual machines?

Part III: Making Threads work hard with Load-Testing!

Well, that was fun! But, though some of the concepts and tradeoffs are the same, threads are not exactly the same and nodes in distributed systems (they share memory). So now let's put threads to work in another way. Read a little bout Locust, and **load generation** → (<https://docs.locust.io/en/stable/running-distributed.html>). In this next set of experiments, you will learn about how to use Locust and how it can load-test your system. Please take more screen shots, and be ready to share your results (and rerun experiments) with your group!

Locust

Locust (<https://docs.locust.io/en/stable/what-is-locust.html>) is a load testing tool written using something called "green threads" (user level threads) in Python. This is going to be a WAY more fun way to test your server set up than just using "curl"! You can get started by:

- Read **write your first test** (<https://docs.locust.io/en/stable/quickstart.html>)
- Read **how to write a locustfile** (<https://docs.locust.io/en/stable/writing-a-locustfile.html#writing-a-locustfile>).

- Run locust master/workers by follow [Run locust in Docker \(https://docs.locust.io/en/stable/running-in-docker.html#running-in-docker\)](https://docs.locust.io/en/stable/running-in-docker.html#running-in-docker). If you are quite new to this, you might want to review how docker compose works.
- Read [Locust's Web UI \(https://docs.locust.io/en/stable/quickstart.html#locust-s-web-interface\)](https://docs.locust.io/en/stable/quickstart.html#locust-s-web-interface) and access it at `localhost:8089` to observe statistics and charts.

Now, it's very little but you DO have a server you can test from the previous homework! Feel free to make it bigger and more exciting, but for now, try 1 worker and 1 user. Look around the Locust UI and take a look at your statistics. To help you understand what percentiles mean, you can read about [why use percentile as metric \(https://www.elastic.co/blog/averages-can-dangerous-use-percentile\)](https://www.elastic.co/blog/averages-can-dangerous-use-percentile). Make sure you have no failures. Try both GET and POST requests.

Do you see any failures? You can make it much spicier if you like by changing things up in your server---depending on your implementations, you might want to review the Collection section from above (in particular the hashmap implementation).

Let's think about tradeoffs here. Which operations will be most common in real world scenario? How does that impact the data structure you are using to save your data?

After you have both requests in place with no failures, capture screenshots showing the difference in statistics between GET and POST. What is going on here? Can you argue reasons behind different numbers? Be ready to have a lively discussions in mock interviews about these results!

Local Test

Let's start to do some load tests! Try different parameters, for example the following parameters:

- GET and POST tasks ratios 3:1
- 1 worker
- 50 users
- 10 users per second ramp up time

Running load tests can put pressure on your machine depending on its specs. We will need to monitor resources usage closely. If your CPU usage is hitting nearly 100% at any point, consider add `wait_time` to your class or decrease the number of user/worker. You can also run `docker stats` to monitor container resources usage.

Amdahl's Law

Record the statistics of GET vs POST with 1 worker. Are the throughputs making sense to you? If we assign more workers to do work, how do you think the throughput will change? Linearly? Let's find out. Increase worker count to 4 and record the statistics again. Don't forget about [Amdahl's Law \(https://en.wikipedia.org/wiki/Amdahl%27s_law\)](https://en.wikipedia.org/wiki/Amdahl%27s_law). Is there any way in which the reading and writing of the same item to something like a hashmap might contribute to your observations?

Context Switching

By default, Locust uses python-requests which can be slow in scenarios where we need high throughput. Read about [FastHttpUser \(https://docs.locust.io/en/stable/increase-performance.html#how-to-use-fasthttpuser\)](https://docs.locust.io/en/stable/increase-performance.html#how-to-use-fasthttpuser) which is C based and designed for high concurrency! In your locustfile,

change `HttpUser` to `FastHttpUser` and run the test again. What do you observe? Document your results, do some research, and try explaining the reasons behind it!

WHAT TO HAND IN AND WHEN

In this assignment, you did a lot of experiments! Please upload your screenshots and observations in a little report, and update your repo with your new code for future fun!

Please post what you liked about Lamports' paper by Wednesday noon Pacific Time, and your report by 9am the following Monday!

Mock Interview Scoring! (2)